

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

Local Storage Systems

Unit 7 • Virtualization and SDDC (10-150-130)



Learning Outcomes

By the end of this unit, you will be able to:



Explain storage virtualization

Describe abstraction, pooling, and thin provisioning, and tell file-level from block-level storage.



Configure local storage

Build LVM, LVM-thin, Directory, and ZFS storage in Proxmox VE and move VM disks between them.



Optimize storage performance

Match RAID layout, cache devices, and VM disk settings to the workload, and measure the result.



Monitor storage health

Track capacity, S.M.A.R.T. data, and ZFS scrubs so problems surface before data is lost.



LESSON 1

Storage Virtualization Concepts

What Storage Virtualization Does

- **Storage virtualization** places a software layer between physical disks and the workloads that use them.
- **Abstraction** — the VM sees an ordinary disk; the host decides where those blocks really live.
- **Pooling** — several disks or partitions are combined into one pool and carved into volumes on demand.
- **Dynamic allocation** — volumes can be created, resized, snapshotted, and moved without touching hardware.



Software-defined storage

When the pooling logic runs in the hypervisor instead of a RAID card or storage array, it is **software-defined**. LVM and ZFS are both software-defined storage running on the Proxmox host itself.

File-Level vs. Block-Level Storage

Proxmox VE sorts every storage type into one of two classes.

File level

- A full POSIX file system — VM disks are files
- Holds any content: disk images, ISOs, templates, backups
- Snapshots come from the `qcow2` image format
- Examples: Directory, NFS, CIFS

Block level

- Each VM disk is its own **raw volume**
- Holds disk images and container volumes only
- Snapshots and clones come from the storage layer
- Examples: LVM, LVM-thin, ZFS zvols, Ceph RBD

ZFS does both: VM disks are stored as block volumes (zvols) and container data as file-system datasets, in the same pool.

Thick vs. Thin Provisioning

- **Thick provisioning** reserves the full size of a disk the moment it is created.
- **Thin provisioning** writes only the blocks the guest actually uses — a 32 GB disk holding 3 GB of data consumes 3 GB.
- Thin storage lets you **over-provision**: promise more disk than you physically own and add capacity later.
- In Proxmox VE, every storage type that supports snapshots also supports thin provisioning.



When a thin pool fills

Every guest with a volume on that storage receives **I/O errors**. File systems can be left inconsistent and data can be corrupted. Over-provision only what you are prepared to monitor.

The Proxmox VE Storage Model

Every storage is an entry in one file, `/etc/pve/storage.cfg`, shared by all nodes in a cluster.

```
dir: local
    path /var/lib/vz
    content iso,vztmpl,backup

lvmthin: local-lvm
    thinpool data
    vgname pve
    content rootdir,images
```

- The first line is `<type>: <storage ID>`; the `content` property limits what the storage may hold.
- Manage entries in **Datacenter → Storage** or with `pvesm` — removing an entry never deletes the data behind it.

LESSON 2

LVM, LVM–Thin, and Directory Storage



LVM: Logical Volume Manager

A lightweight layer that pools disks and divides the space into volumes.

Layer	What it is	Command to inspect
Physical volume (PV)	A disk or partition handed to LVM	pvs
Volume group (VG)	A pool built from one or more PVs	vgs
Logical volume (LV)	A slice of the VG — one per VM disk	lvs

- Plain LVM is **thick**: each logical volume reserves its full size when created.
- It can be placed on a shared iSCSI or Fibre Channel LUN, which makes it the usual choice on a SAN.

LVM-Thin

- A **thin pool** is a special logical volume that allocates blocks only as guests write them.
- Supports efficient **snapshots** and **linked clones** — plain LVM has neither natively.
- The default `local-lvm` storage is a thin pool named `data` in volume group `pve`.
- Thin pools **cannot be shared** across nodes — LVM-thin is local storage only.



Watch two numbers

`lvs` reports **Data%** and **Meta%** for a thin pool. Either one reaching 100% stops writes for every guest on it.

Building LVM and LVM-Thin Storage

The same three layers, created by hand. Replace the device with your spare disk.

```
pvcreate /dev/sdX                # hand the disk to LVM
vgcreate vmdata /dev/sdX         # build a volume group
pvesm add lvm lvm-store --vgname vmdata # register it as thick LVM storage

lvcreate -L 100G -n thinpool vmdata # carve out a volume
lvconvert --type thin-pool vmdata/thinpool # convert it to a thin pool
pvesm add lvmthin thin-store --vgname vmdata --thinpool thinpool
```

GUI equivalent: Node → Disks → LVM or LVM-Thin → Create, which also adds the storage entry for you.

Directory Storage

Any mounted file system can be a storage. Proxmox VE lays content out in fixed sub-directories:

Content type	Sub-directory	Typical files
VM disk images	images/<VMID>/	vm-100-disk-0.qcow2
ISO images	template/iso/	Installer media
Container templates	template/cache/	Template archives
Backups	dump/	vzdump archives

- Image formats are `raw` and `qcow2`; only `qcow2` gives a directory storage snapshots and thin provisioning.
- The built-in `local` storage is a directory at `/var/lib/vz`.

Choosing a File System: ext4 vs. XFS

When you create a Directory storage on a new disk, you pick the file system underneath it.

ext4

- Mature, widely supported, the Proxmox VE installer default
- Journaling for crash recovery; extent-based allocation
- Can be grown and shrunk
- A safe general-purpose choice

XFS

- Built for large files and high, parallel throughput
- Delayed allocation and online defragmentation
- Can be grown but never shrunk
- Suits large disk images and busy hosts

What both lack: Neither file system checksums your data or manages multiple disks. For that, the next lesson turns to ZFS.

Local Storage Types Compared

Four ways to store a VM disk on local hardware — as of Proxmox VE 9.2.

Storage	Level	Thin provisioning	Snapshots	Best fit
LVM	Block	No	Volume chains (preview)	Simple or SAN-backed disks
LVM-thin	Block	Yes	Yes	Space-efficient local VMs
Directory	File	With qcow2	With qcow2	ISOs, backups, any content
ZFS	Both	Yes	Yes	Integrity and redundancy

None of these is shared: a VM on local storage must have its disks copied to migrate to another node.

What Changed: Snapshots as Volume Chains

- Since **Proxmox VE 9**, thick LVM, Directory, NFS, and CIFS storage can take VM snapshots as a **volume chain**.
- Taking a snapshot freezes the current volume under the snapshot name and starts a new volume that records only the changes.
- Enabled per storage with `snapshot-as-volume-chain`; it applies only to disks created afterwards.
- It avoids native LVM snapshots, which slow every write in the volume group while they exist.



Still a technology preview

In 9.2 the feature is not yet production-ready. A VM with volume-chain snapshots on local storage cannot be migrated, and the newest snapshot cannot be removed while the VM runs.

LESSON 3

ZFS for Virtualization



What Makes ZFS Different

- **Volume manager and file system in one** — ZFS manages the disks, the redundancy, and the data.
- **Copy-on-write** — changed blocks are written to new locations, so snapshots are instant and nearly free.
- **Checksums on every block** — corruption is detected on read and repaired from a redundant copy.
- **Built in:** compression, thin provisioning, snapshots, clones, and send/receive replication.



Give ZFS the raw disks

Do not put ZFS on a hardware RAID controller that manages its own cache. ZFS must talk to the disks directly — use an **HBA** or a controller in pass-through mode.

ZFS RAID Levels

A pool is built from **vdevs**; ZFS stripes data across every vdev in the pool.

Layout	Min. disks	Usable capacity	Survives
Mirror (RAID1)	2	50%	One disk per mirror
Striped mirrors (RAID10)	4	50%	One disk per mirror
RAIDZ-1	3	N - 1 disks	Any one disk
RAIDZ-2	4	N - 2 disks	Any two disks
RAIDZ-3	5	N - 3 disks	Any three disks

RAIDZ-2 examples: four disks give 50% usable space, six disks give 67%. Since Proxmox VE 9, a RAIDZ vdev can be widened one disk at a time.

Mirrors or RAIDZ for VM Workloads?

For running VMs, IOPS usually matters more than raw throughput.

Mirror vdevs

- Each mirror writes like one disk; reads scale with the number of disks
- Add more mirrors and IOPS rises with them
- Fast, simple resilver after a disk failure
- Cost: half the raw capacity

RAIDZ vdevs

- A whole vdev delivers roughly the IOPS of one disk
- High sequential throughput and more usable capacity
- VM disks on RAIDZ consume extra space for parity
- Better suited to backups and bulk data

Proxmox guidance: Use mirror vdevs for VM storage unless your environment has a specific need that RAIDZ performance can satisfy.

ZFS and Memory: The ARC

- The **Adaptive Replacement Cache (ARC)** keeps recently and frequently read blocks in RAM.
- Plan for at least **8 GB** of host memory before using ZFS; more is better.
- Sizing rule of thumb for the ARC: **2 GiB base + 1 GiB per TiB** of pool storage.
- **ECC memory** is recommended — ZFS trusts what RAM tells it.



Default ARC limit

New installs cap the ARC at **10% of host RAM, up to 16 GiB**. Change it with `zfs_arc_max` in `/etc/modprobe.d/zfs.conf`.

Accelerating a Pool with SSDs

Three optional device classes, each solving a different problem.

Device	What it speeds up	Requirement
Log (SLOG)	Synchronous writes, by moving the ZFS intent log (ZIL) off the pool	SSD with power-loss protection
Cache (L2ARC)	Random reads of mostly static data, as a second cache tier	Fast SSD; no redundancy needed
Special	Metadata, and optionally small blocks, on HDD pools	Same redundancy as the pool

Losing a log or cache device is survivable. Losing a special device loses the pool, and adding one cannot be undone.

Compression and Deduplication

Compression — leave it on

- `lz4` is the recommended algorithm: very little CPU overhead
- Less data written can make I/O faster, not slower
- Applies only to blocks written after it is enabled
- `zfs get compressratio <pool>` shows the saving

Deduplication — leave it off

- Every write is checked against a table of existing blocks
- That table needs memory, and lookups slow writes
- Fast Dedup in OpenZFS 2.3 reduces the cost but does not remove it
- Worth it only for highly duplicate data

Test before enabling dedup: It is set per dataset, but its table lives in the pool and remains until the deduplicated data is rewritten or destroyed.

Creating a ZFS Pool

A two-disk mirror, compressed, and registered as thin-provisioned VM storage.

```
zpool create -f -o ashift=12 tank mirror /dev/sdX /dev/sdY
zfs set compression=lz4 tank
zfs create tank/vmdata
pvesm add zfspool zfs-store --pool tank/vmdata --sparse 1 --content images,rootdir

zpool status tank          # health and layout
zfs list                   # datasets and space used
```

GUI equivalent: Node → Disks → ZFS → Create: ZFS — choose the RAID level, compression, and ashift in one dialog.

ZFS in Proxmox VE 9.2

Proxmox VE 9.2 (May 2026) ships ZFS 2.4. What is new since the 8.x releases:

Feature	What it does	Status
RAIDZ expansion	Adds a disk to an existing RAIDZ vdev with zpool attach	Available since PVE 9.0
Fast Dedup	A faster redesign of the deduplication table	Available; still use sparingly
Direct I/O	Lets reads and writes bypass the ARC on fast NVMe	Available
Native encryption	Encrypts datasets and zvols at rest	Experimental in Proxmox VE

Enabling new pool features with zpool upgrade is one-way: older ZFS versions can no longer import the pool.

LESSON 4

Performance and Monitoring



What Determines Storage Performance

Each layer between the guest and the disk adds its own limits.

Factor	Options	Effect
Media	HDD, SATA/SAS SSD, NVMe	Sets the IOPS and latency ceiling
Redundancy layout	Mirror, RAIDZ, hardware RAID	Trades write speed for capacity
Storage type	LVM, LVM-thin, qcow2 file, zvol	Adds allocation and copy-on-write cost
Guest disk settings	Bus, cache mode, I/O thread	Controls how the VM reaches the storage

Measure IOPS for VM workloads and throughput for backups and bulk copies — they are different questions.

VM Disk Settings That Matter

- **Bus** — use **VirtIO SCSI** for the best performance and the widest feature support.
- **Discard** — passes guest TRIM down to thin storage so deleted space is actually returned.
- **IO thread** — gives the disk its own I/O thread instead of sharing the main one.
- **Cache** — the default, `none`, is the safe choice; `writeback` is faster but can lose recent writes on power failure.



Reclaim thin space

Enable **Discard** on the disk, then run `fstrim -av` inside a Linux guest. Watch the pool usage fall on the host.

Benchmarking with fio

One test, repeated on each storage type, gives numbers you can compare.

```
apt install fio

fio --name=randrw --filename=/mnt/pve/dir-store/fio.dat --size=1G \
    --rw=randrw --bs=4k --ioengine=libaio --direct=1 --iodepth=32 \
    --runtime=60 --time_based --group_reporting

pveperf /mnt/pve/dir-store          # quick built-in check, including fsyncs/second
```

- `--bs=4k --rw=randrw` models VM-style random I/O; `--direct=1` bypasses the page cache so you test the disk, not RAM.
- Record IOPS, throughput, and latency for each storage type, and keep every other setting identical.

Monitoring Storage Health

Three signals, three places to look.

Signal	What it tells you	Where to check
S.M.A.R.T.	Wear, reallocated sectors, and errors the disk reports about itself	Node → Disks, or smartctl -a
ZFS scrub	Reads every block, verifies checksums, repairs from redundancy	zpool scrub, then zpool status
ZFS events (ZED)	Emails on pool errors and degraded devices	/etc/zfs/zed.d/zed.rc

Capacity: Know Before It Fills

- `pvesm status` lists every storage with total, used, and available space.
- For thin pools, compare what is **provisioned** with what is **physically used** — `lvs` and `zfs list` show both.
- Keep ZFS pools **below about 80% full**; copy-on-write needs free space to stay fast.
- Snapshots hold on to old blocks. Forgotten snapshots are the most common cause of a pool that fills unexpectedly.



Set a threshold and act on it

Decide in advance what usage triggers action — adding disks, removing snapshots, or moving VMs with **Disk Action → Move Storage**.

Security Focus: Data Left Behind

- Deleting a VM disk on thick LVM frees its extents but **does not erase them**.
- The next volume given those extents can read the old data. **Wipe Removed Volumes** (`saferemove`) zeroes a volume on deletion.
- Thin provisioning is an **availability** risk: one runaway guest can fill the pool for all of them.
- Retired disks still hold VM images. Wipe or destroy them before they leave the building.



SECURITY FOCUS

Storage is where every guest's data meets. Decide who may allocate space, how deleted volumes are handled, and how full a pool may get.

Why it matters: A single unwiped volume can expose one tenant's data to the next.

Hands-On Lab: Local Storage Configuration

NETLAB · Lab 8 — Local Storage Configuration and Management · Proxmox VE

- Create LVM and LVM-thin storage and confirm thin allocation.
- Add a Directory storage on ext4 or XFS.
- Build a ZFS pool and enable compression.
- Move a virtual machine disk between storage types.
- Compare space utilization across the four storage types.



See Blackboard for the lab assignment details and what to submit.

Key Takeaways



Storage virtualization pools disks and hands out volumes; thin provisioning is a loan you must monitor.



LVM-thin is the efficient local default; Directory storage holds any content; plain LVM can be shared.



ZFS combines RAID, volume management, and checksums — use mirror vdevs for VM workloads.



Benchmark with identical tests, and watch capacity, S.M.A.R.T. data, and scrubs.



Deleted is not erased: wipe removed volumes where tenants share storage.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 7 · Local Storage Systems